RUNGTA COLLEGE
RUNGTA EDUCATIONAL FOUNDATION
BHILAI-CHHATTISGARH
Redefining Education's Future

Python Report

Project: Word Counter & Text Analyzer

Student Name	Rupesh Kumar Sidar
ERP No.	RU 01065
College	Rungta International Skills University
Language	Python 3
Concepts	Strings, Dictionaries, Sets, Frequency Analysis

1. Problem Definition

Build a tool that analyses a piece of text, typed by the user or read from a file, and reports the number of words, characters and sentences, along with the frequency of each word. Counting is case-insensitive and punctuation is removed before counting words.

2. Objectives

• Read text from the user or from a text file.
• Clean the text using string methods (lower, split, strip, replace).
• Count word frequency using a dictionary.
• Find unique words using a set and show results sorted by frequency.
• Handle invalid input (empty text, missing file) with clear messages.

3. Approach and Logic

Step 1 - Input. The user selects whether to type text or enter a file name. Empty text prints *No text to analyse.* and a missing file prints *File 'name' not found.*

Step 2 - Cleaning. The text is converted to lower case with `lower()`. Apostrophes are removed and all other punctuation is replaced by a space using `replace()`. The result is split into a list of words with `split()`.

Step 3 - Sentence count. The characters `!` and ? are replaced by `.`; the text is split on `.` and non-empty parts are counted.

Step 4 - Frequency. A dictionary maps each word to its count.

Step 5 - Unique words. A set built from the word list removes duplicates.

Step 6 - Sorting and output. Words are sorted by count (high to low) and then alphabetically using `sorted()`.

4. Algorithm

```
1. START
2. Show menu (1 Summary, 2 Frequency, 3 Unique words, 4 Exit)
3. For options 1-3 get text (typed or from file)
      if text empty      -> print "No text to analyse."
      if file not found  -> print "File '<name>' not found."
4. words     = clean(text)          # lower, remove punctuation, split
5. chars     = len(text)
6. sentences = count of non-empty parts split on . ! ?
7. freq      = {} ; for each word: freq[word] += 1
8. unique    = set(words)
9. Print results (sorted by frequency)
10. Repeat from step 2 until Exit
11. STOP
```

5. Program Structure

Function	Purpose
read_text_from_user()	Takes text typed by the user
read_text_from_file()	Reads a file; returns an error message if not found
clean_words(text)	Lower-cases, removes punctuation and returns the word list
count_sentences(text)	Counts sentences ending in . ! or ?
word_frequency(words)	Builds the word-to-count dictionary
sorted_by_frequency(freq)	Sorts by count (desc), then alphabetically
analyse(text)	Runs all calculations; returns a result dictionary
show_summary / show_frequency / show_unique	Print the reports
get_text() / main()	Input handling and the menu loop

6. Test Cases and Results

#	Input	Expected Output	Actual Output	Result
1	Python is easy. Python is fun.	Words: 6, Characters: 30, Sentences: 2	Words: 6, Characters: 30, Sentences: 2	Pass
2	Hello	Words: 1, Characters: 5, Sentences: 1, Unique: 1	Words: 1, Characters: 5, Sentences: 1, Unique: 1	Pass
3a	(empty text)	No text to analyse.	No text to analyse.	Pass
3b	File notes.txt (missing)	File 'notes.txt' not found.	File 'notes.txt' not found.	Pass
4	The cat saw the dog. The dog ran. (option 2)	the: 3, dog: 2; Unique words: 5 (cat, dog, ran, saw, the)	the: 3, dog: 2, cat: 1, ran: 1, saw: 1; Unique words: 5 (cat, dog, ran, saw, the)	Pass

7. Sample Output

```
=== Word Counter & Text Analyzer ===
Menu: 1. Analyse new text  2. Word frequency  3. Unique words  4. Exit

Choose an option: 1
Input source:  1. Type text   2. Read from file
Choose 1 or 2: 1
Enter your text: Python is easy. Python is fun.
Words: 6 | Characters (with spaces): 30 | Sentences: 2 | Unique words: 4

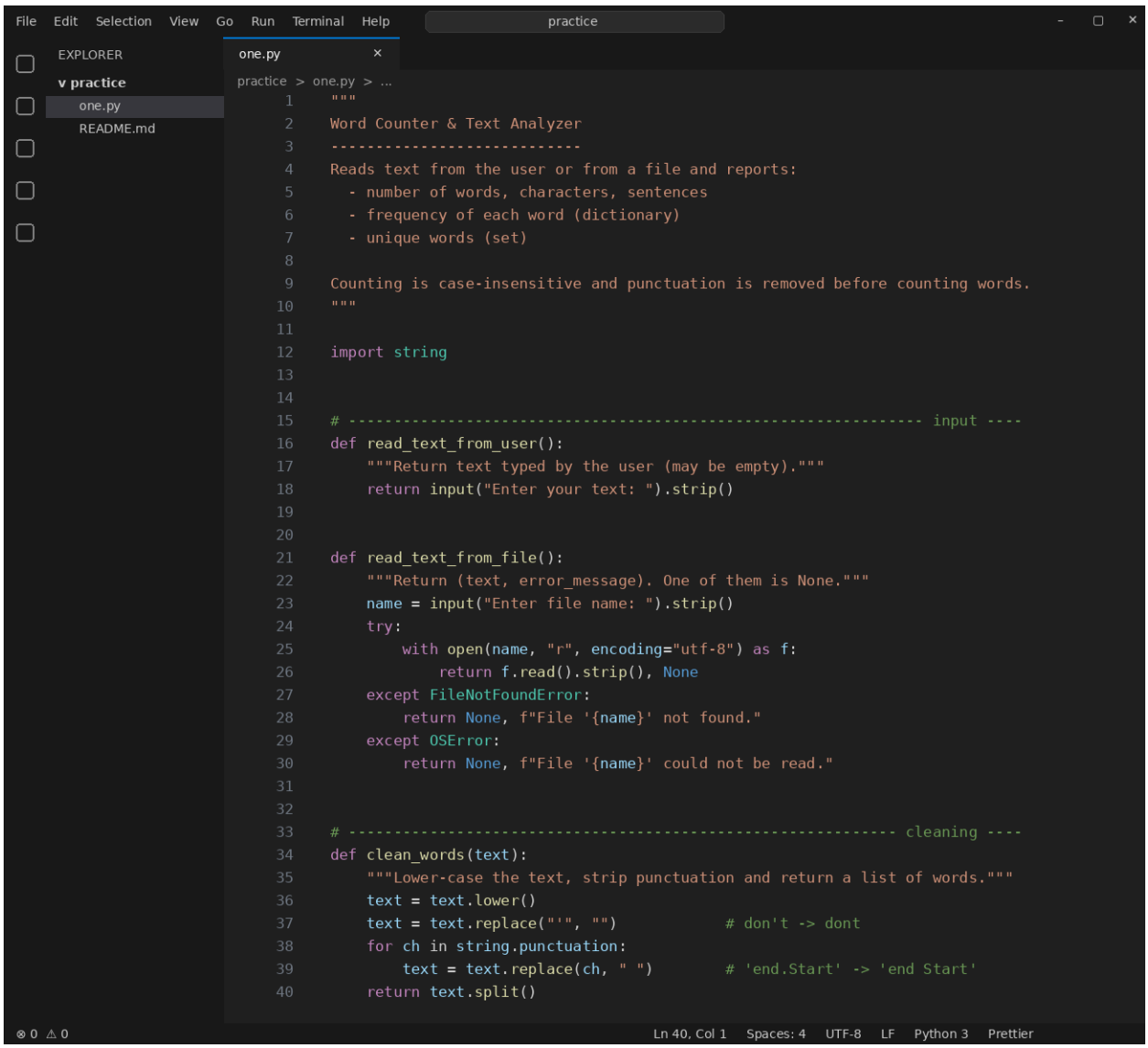
Choose an option: 2
Top words: is: 2 | python: 2 | easy: 1 | fun: 1
Unique words: 4 (easy, fun, is, python)
```

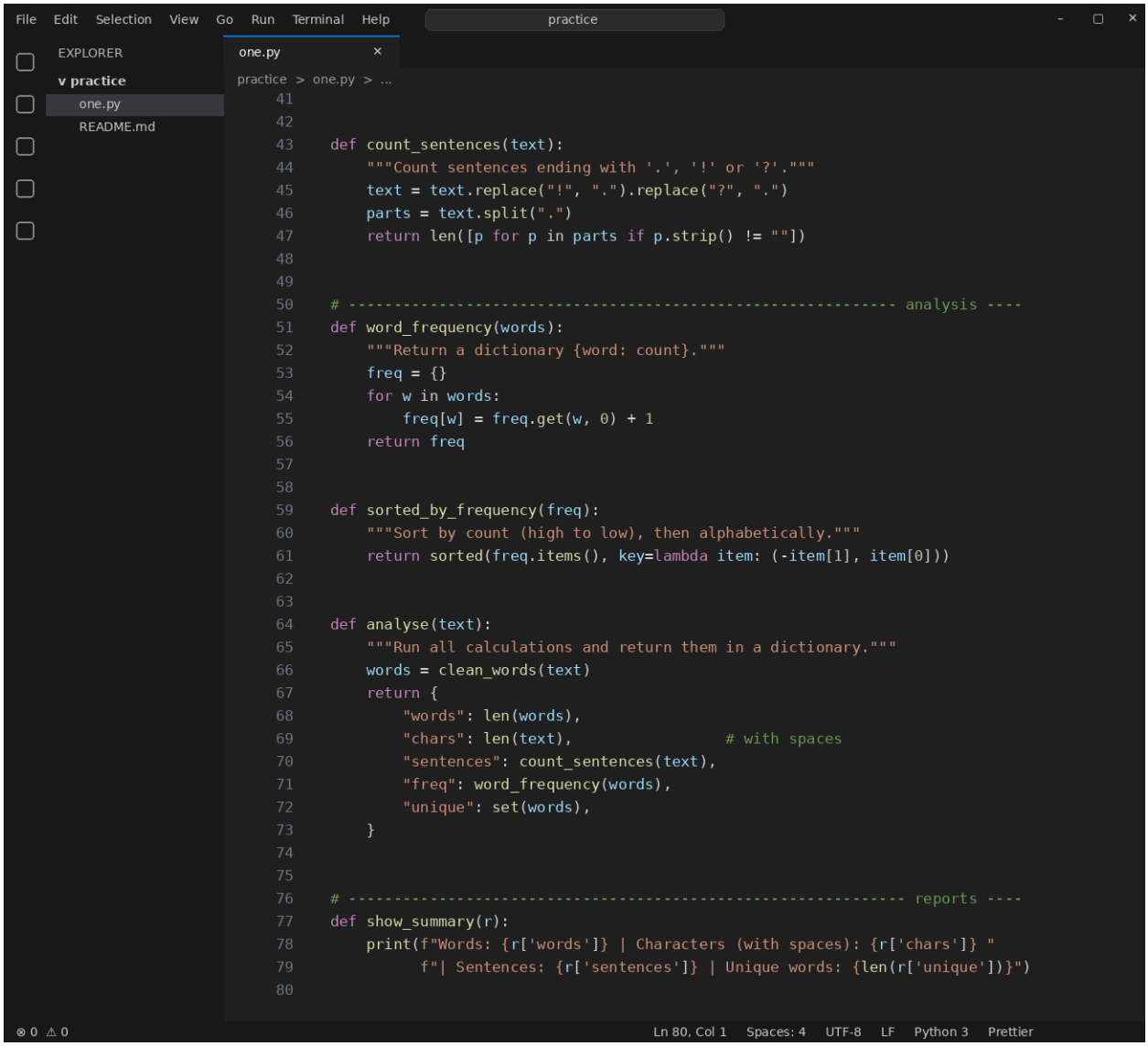
8. Conclusion

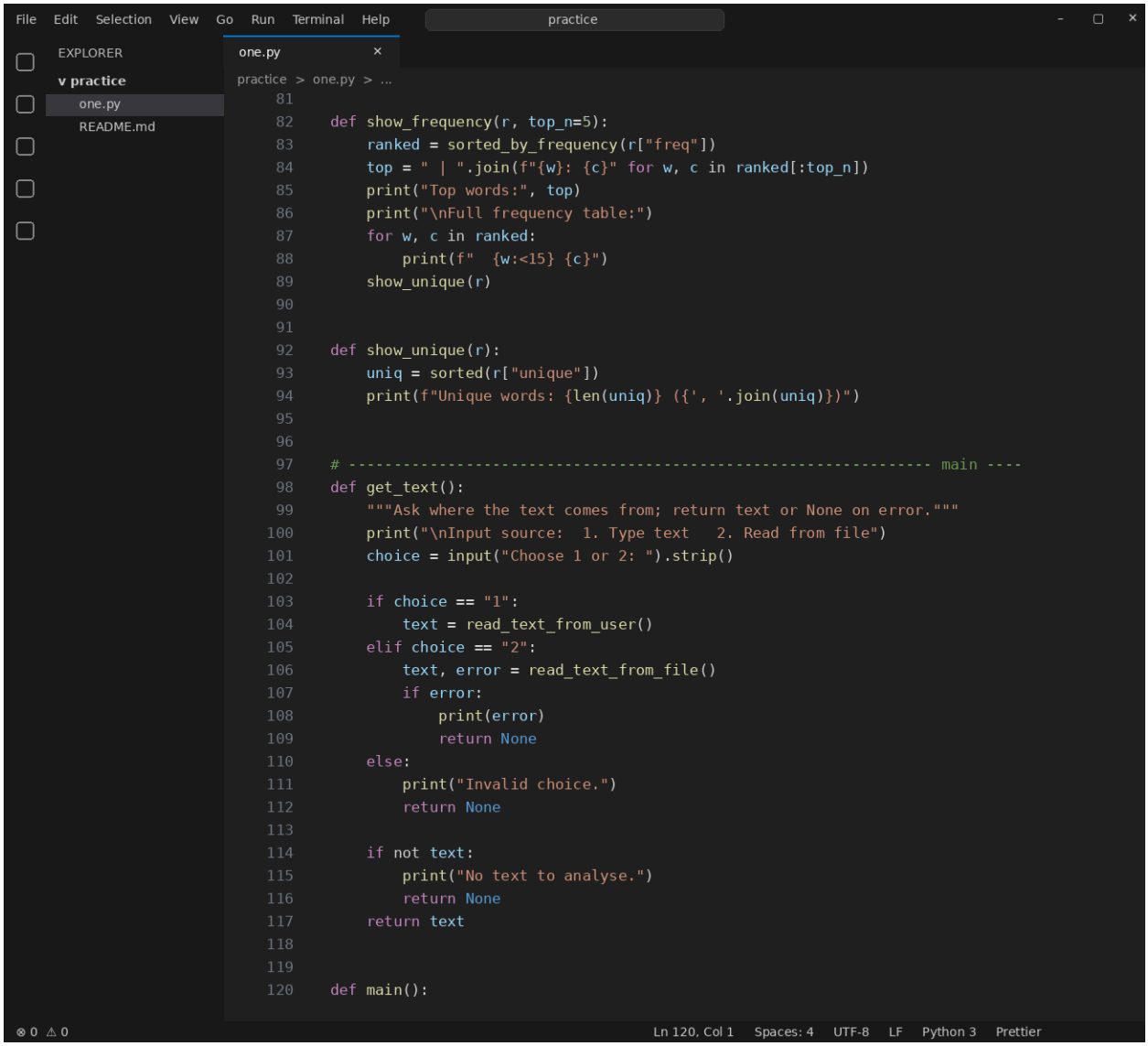
The Word Counter & Text Analyzer meets all the bare minimum requirements. It reads text from the user or a file, cleans it using string methods, counts word frequency with a dictionary, finds unique words with a set and displays results sorted by frequency. All four test cases produce the expected output. Possible future improvements include ignoring common stop words, exporting results to a file and a graphical interface.

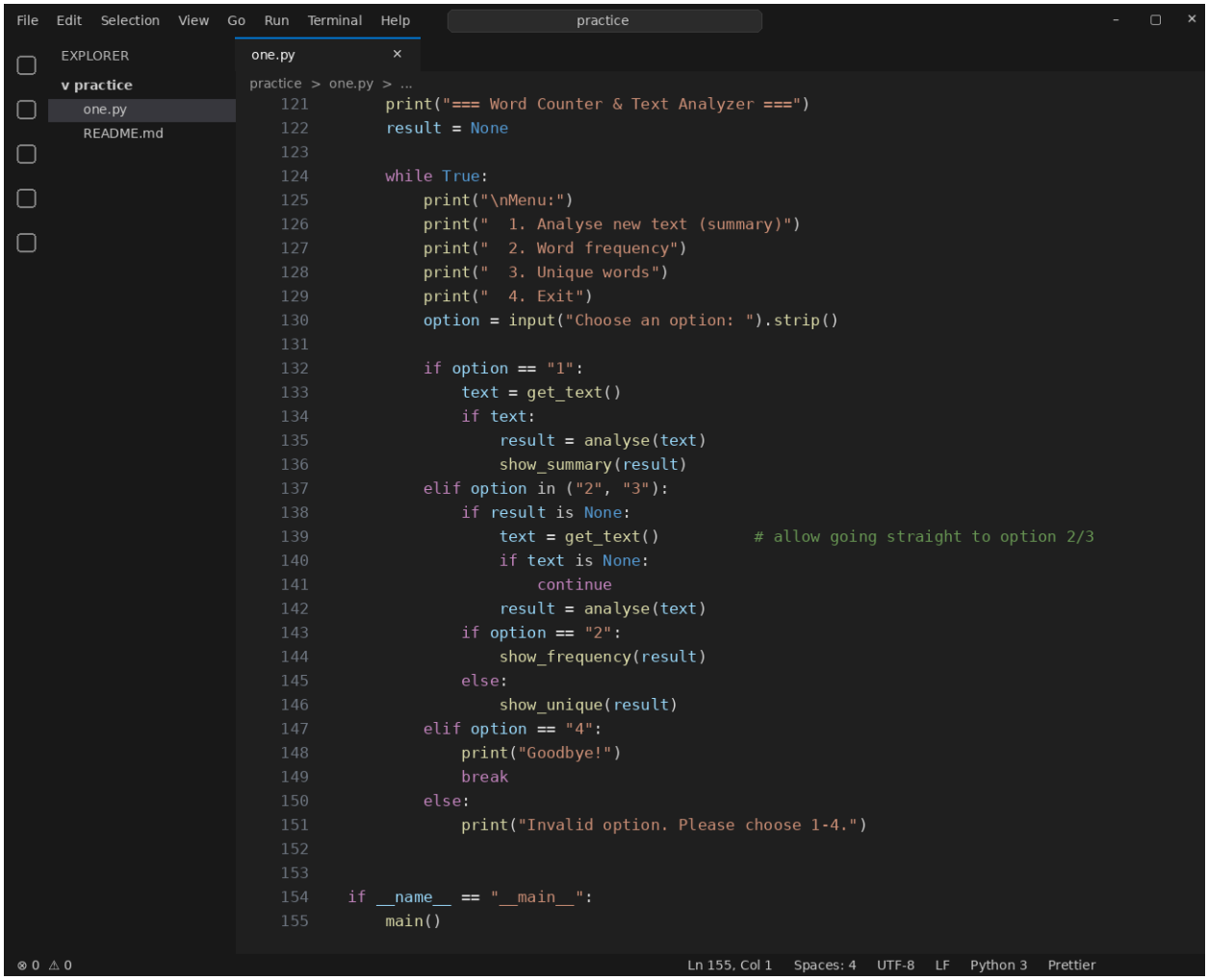
9. Source Code (VS Code Screenshots)

Screenshots of the program (one.py) in Visual Studio Code:




File Edit Selection View Go Run Terminal Help
practice
EXPLORER
v practice
one.py
README.md
one.py
practice > one.py > ...
41
42
43 def count_sentences(text):
44 """Count sentences ending with '.', '!' or '?'."""
45 text = text.replace("!", ".").replace("?", ".")
46 parts = text.split(".")
47 return len([p for p in parts if p.strip() != ""])
48
49
50 # -- analysis ----
51 def word_frequency(words):
52 """Return a dictionary {word: count}."""
53 freq = {}
54 for w in words:
55 freq[w] = freq.get(w, 0) + 1
56 return freq
57
58
59 def sorted_by_frequency(freq):
60 """Sort by count (high to low), then alphabetically."""
61 return sorted(freq.items(), key=lambda item: (-item[1], item[0]))
62
63
64 def analyse(text):
65 """Run all calculations and return them in a dictionary."""
66 words = clean_words(text)
67 return {
68 "words": len(words),
69 "chars": len(text), # with spaces
70 "sentences": count_sentences(text),
71 "freq": word_frequency(words),
72 "unique": set(words),
73 }
74
75
76 # -- reports ----
77 def show_summary(r):
78 print(f"Words: {r['words']} | Characters (with spaces): {r['chars']} "
79 f"| Sentences: {r['sentences']} | Unique words: {len(r['unique'])}")
80
⊗ 0 ⚠ 0
Ln 80, Col 1 Spaces: 4 UTF-8 LF Python 3 Prettier


File Edit Selection View Go Run Terminal Help
practice
EXPLORER
v practice
one.py
README.md
one.py
practice > one.py > ...
81
82 def show_frequency(r, top_n=5):
83 ranked = sorted_by_frequency(r["freq"])
84 top = " | ".join(f"{w}: {c}" for w, c in ranked[:top_n])
85 print("Top words:", top)
86 print("\nFull frequency table:")
87 for w, c in ranked:
88 print(f" {w:<15} {c}")
89 show_unique(r)
90
91
92 def show_unique(r):
93 uniq = sorted(r["unique"])
94 print(f"Unique words: {len(uniq)} ({', '.join(uniq)})")
95
96
97 # ---------- main ----
98 def get_text():
99 """Ask where the text comes from; return text or None on error."""
100 print("\nInput source: 1. Type text 2. Read from file")
101 choice = input("Choose 1 or 2: ").strip()
102
103 if choice == "1":
104 text = read_text_from_user()
105 elif choice == "2":
106 text, error = read_text_from_file()
107 if error:
108 print(error)
109 return None
110 else:
111 print("Invalid choice.")
112 return None
113
114 if not text:
115 print("No text to analyse.")
116 return None
117 return text
118
119
120 def main():
⊗ 0 ⚠ 0
Ln 120, Col 1 Spaces: 4 UTF-8 LF Python 3 Prettier


File Edit Selection View Go Run Terminal Help
practice
EXPLORER
v practice
one.py
README.md
one.py
practice > one.py > ...
121 print("=== Word Counter & Text Analyzer ===")
122 result = None
123
124 while True:
125 print("\nMenu:")
126 print(" 1. Analyse new text (summary)")
127 print(" 2. Word frequency")
128 print(" 3. Unique words")
129 print(" 4. Exit")
130 option = input("Choose an option: ").strip()
131
132 if option == "1":
133 text = get_text()
134 if text:
135 result = analyse(text)
136 show_summary(result)
137 elif option in ("2", "3"):
138 if result is None:
139 text = get_text() # allow going straight to option 2/3
140 if text is None:
141 continue
142 result = analyse(text)
143 if option == "2":
144 show_frequency(result)
145 else:
146 show_unique(result)
147 elif option == "4":
148 print("Goodbye!")
149 break
150 else:
151 print("Invalid option. Please choose 1-4.")
152
153
154 if __name__ == "__main__":
155 main()
⊗ 0 ⚠ 0
Ln 155, Col 1 Spaces: 4 UTF-8 LF Python 3 Prettier